

05 · Inter-Task-Kommunikation und Interrupts

Queues, Mailboxes und ISR-Anbindung · EZS · DHBW Stuttgart

Prof. Dr.-Ing. Daniel Klünder

Folie

1 / 48

Willkommen zurück zu Echtzeitsysteme!

- Fünfter Termin: Daten transportieren, nicht nur sperren
- Ein FreeRTOS-Kernel auf dem CM4, wie in VL3/VL4, sofern das Projekt das bestätigt
- Eine lokale Queue ist kein Kanal zwischen zwei getrennten Kernen
- DMA und der zweite Kern bleiben ein kurzer Anhang

Notizen

1 / 48

Keine .ioc im Kursrepo. UART, EXTI-Zielkern, Tick und NVIC-Gruppierung am Cube-Projekt lesen. CMSIS-Spezifikation, ST-Adapter und native API nicht vermischen.

Folie

2 / 48

Agenda für heute

1. Historie gegen aktuellen Wert
2. Queue: Ring, Warten, Überlauf
3. Kopie, Pointer, Pufferkreislauf
4. Mailbox und mehrere Leser
5. Top-Half, zulässige ISR-API, EXTI-Labor

Notizen

2 / 48

Agendanotiz bleibt auf dieser Folie: Put am Tail, Get am Head. Nachrichtenvorrang ist nicht die Taskpriorität.

Recap: Die Lehren aus Termin 4

- `counter++` ist ein Read-Modify-Write, nicht universell drei Befehle
- Präemption auch durch Ereignisse, Wechsel im Port über PendSV
- PIP kürzt die Extra-Zeit durch Medium, nicht jede Verzögerung
- Ein Mutex kann einen konsistenten Snapshot schützen
- Er liefert keine Historie und keine Ereignisschlange

Notizen

3 / 48

Verlorenes Update endete bei 6 statt 7. Das bleibt wahr, auch wenn der Compiler mehr als drei Befehle erzeugt.

Folie

4 / 48

Mutexe schützen, aber transportieren nicht!

- Viele Samples hintereinander: der letzte Wert überschreibt die früheren
- „Mehr Zeit am Schloss als beim Rechnen“ nur mit Messung oder als Hypothese
- Eine Queue kopiert und weckt; auch das kostet
- Hohe Raten oft als Block, nicht jedes Sample als eigene Nachricht
- 44000 Samples/s nicht als belegte Laborzahl verwenden

Notizen

4 / 48

Backpressure kann eine Softwarequelle bremsen. ADC, UART-Empfang oder DMA laufen unter Umständen weiter.

Motivation: Producer & Consumer

Fünf Samples bei $t = 1, 2, 3, 4, 5$ ms. Der Leser kommt nach dem fünften.

Speicher	Inhalt
eine globale Variable	nur Sample 5
FIFO mit Platz	Folge 1 bis 5, wenn nichts verworfen wurde

- „Alle Ereignisse in Reihenfolge“: Überschreiben ist Verlust
- „Aktueller Zustand“: Überschreiben kann gewollt sein
- Sequenznummer und Zeitstempel machen Lücken und Alter sichtbar
- Die Zeitquelle muss zur Einheit passen

Notizen

5 / 48

Frage: Braucht ein Display jeden Zwischenwert? Oft den letzten gültigen. Braucht ein Tasterzähler jeden Druck? Ja, sonst fehlt ein Ereignis. Die Anforderung entscheidet, nicht die Queue an sich.

Folie

6 / 48

Die Falle der globalen Variable

- Geschützt: ein konsistenter Stand
- Ungeschützt: zerrissene Felder, wie in VL4
- Kein Warten auf „neue Daten“, kein Rückstau, keine Reihenfolge
- Ein Puffer gleicht Bursts aus
- Er macht einen dauerhaft langsameren Consumer nicht schneller

Notizen

6 / 48

Durchsatz μ unter der Ankunft λ füllt jeden endlichen Puffer.

Die Lösung: Inter-Task Communication (ITC)

Begrenzter Puffer, nicht nur zwei Kästen:

Producer → Slot → Slot → Slot → Consumer

- Put reiht eine **Kopie** des Elements ein
- Get entnimmt ein Element
- Mehrere Consumer teilen sich die Arbeit
- Sie bekommen nicht alle dieselbe Nachricht

Notizen

7 / 48

GetCount und GetSpace sind Momentaufnahmen, keine Reservierung für den nächsten Aufruf.

Folie

8 / 48

Lernziele des heutigen Tages

1. Historie und Latest-Value auseinanderhalten
2. Einen Ring mit Head, Tail und Count lesen
3. Wertkopie, Pointerkopie und Pufferbesitz trennen
4. Put mit Timeout 0 nicht mit Overwrite verwechseln
5. EXTI nur mit zulässiger NVIC-Priorität an den Kernel hängen

Notizen

8 / 48

Ready nach einem Put ist noch nicht Running.

Folie

9 / 48

Die Queue (Warteschlange)

- Im Adapter mit `msg_prio = 0`: normales Put/Get ist FIFO
- Die CMSIS-Spezifikation sortiert nach `msg_prio`
- Der dokumentierte CMSIS-FreeRTOS-Adapter **ignoriert** den Parameter
- Das ist keine Taskpriorität
- „Immer 0, historisch“ reicht als Erklärung nicht

Notizen

9 / 48

Version im Projekt prüfen. Öffentliches `main` ist nicht die Laborfassung. Quelle: CMSIS Message Queue

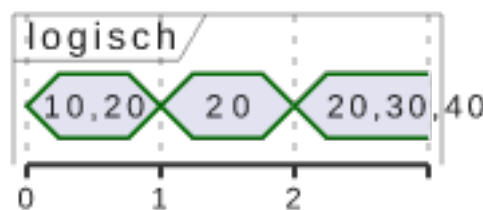
Der Lebenszyklus einer Queue

Ring $N = 3$. Put 10, Put 20, Get liefert 10, Put 30, Put 40.

Logische Restfolge: **20, 30, 40**.

Physisch danach: Slots $[40, 20, 30]$, Head = Tail = 1, Count = 3.

Gleiche Indizes können leer oder voll sein. Count entscheidet.



Put am Tail, Get am Head. Wrap ist Absicht. Synthetisches Bild, kein Boardtrace.

Speichermanagement einer Queue

- Nutzdaten, Kontrollblock, Alignment, Allokator getrennt zeichnen
- $N = 10$ und `uint32_t`: **40 Byte** Nutzdaten, plus Verwaltung
- `osMessageQueueNew` mit Attribut `NULL`: dynamisch, kann `NULL` liefern
- `NULL` ist nicht nur „Heap voll“: auch Parameter, Kontext, Konfiguration
- Statische Puffer: Größe aus Header, Adapter und Linker-Map
- Delete erst, wenn niemand mehr nutzt
- Ein gespeicherter Pointer gibt das Bildarray nicht frei

CMSIS-New und `xQueueCreateStatic` nicht gleichsetzen. Keine universelle Gesamtgröße neben den 40 Byte behaupten.

Die Magie: Blockieren bei Leerlauf

- Consumer, Timeout warten, Queue leer → Blocked
- Put macht ihn Ready
- Er läuft, wenn der Scheduler ihn wählt
- `timeout = 0`: sofortiger Status, Zielwert nicht als neue Nachricht behandeln
- Endliche Wartezeit kann mit Timeout enden
- `osWaitForever` macht einen ungültigen Handle nicht gültig

Notizen

12/48

Blocked: keine eigenen Befehle. Tick und IRQs kosten weiter. Nicht „CPU 0 %“.

Folie

13/48

Die Magie: Blockieren bei Überlast

- Producer, Queue voll, Timeout > 0 → Blocked, bis ein Platz frei wird
- Try-Put: nichts eingereicht, Nachricht bleibt beim Sender
- Nicht jeden Fehler „Queue voll“ nennen
- Nur bei `osOK` weiterverarbeiten

```
osStatus_t st = osMessageQueuePut(q, &msg, 0, 0);  
if (st != osOK) {  
    drop_count++; /* nicht eingereicht */  
}
```

Notizen

13/48

Hält der Producer dabei einen Mutex, den der Consumer vor dem Get braucht, entsteht der Zyklus aus VL4. Queue-Threadsicherheit verhindert den Deadlock nicht.

Zusammenfassung: ITC-Vorteile

- Reihenfolge, solange Platz und Verträge passen
- Warten ohne Polling
- Kopie entkoppelt die Stackvariable
- Keine Priority Inheritance für das Paar Producer/Consumer
- Kein unendlicher Puffer bei $\lambda > \mu$

Notizen

14 / 48

Vorteil ist Struktur und ein klarer Überlauf, nicht automatische Frist.

Folie

15 / 48

Überlauf: Burst und Dauerlast

Lehrannahmen: $N = 10$, anfangs leer, API-Zeit 0. Put bei 1, 2, 3, ... ms. Get bei 5, 10, 15, ... ms. Zur selben Zeit zuerst Put, dann Get. Voller Try-Put verwirft den **neuen** Wert.

Zeitpunkt	Stand
nach 5 ms	Count 4
nach 10 ms	Count 8
bei 12 ms	Count 10
erster Fehl-Put	13 ms

Über 1–20 ms: 20 Versuche, **13** erfolgreiche Puts, **7** verworfen, **4** Gets, **9** bleiben.

$\lambda = 1000/\text{s}$, $\mu = 200/\text{s}$: kein endlicher Puffer hält das verlustfrei, wenn jede Entnahme nur eine Nachricht ist. Ein Consumer, der danach einen ganzen Stapel leert, ist ein anderes Modell.

Notizen

15 / 48

Bilanz: $20 = 13 + 7$ und $13 = 4 + 9$. Der Erfolg bei $t = 16$ ms kommt, weil das Get bei 15 einen Platz gemacht hat und der Put bei 15 selbst noch scheiterte. Synthetisch.

Kapazität planen

Unter einem **begrenzten** verlustfreien Modell:

$$N \geq \max_t (A(t) - D(t))$$

- A : angebotene Nachrichten, D : Entnahmen
- Nicht erst die Puts zählen, die nach Drops noch klappen
- Fünf Nachrichten ohne zwischenzeitliches Get brauchen fünf Plätze
- Ein schon verarbeiteter Satz liegt außerhalb der Queue
- Payload-Pool extra zählen

Entscheidung	Wirkung
Quelle drosseln	nur wenn die Quelle warten kann
neuen Wert verwerfen	Lücke in der Historie
alten ersetzen	Latest-Value, andere API
Fehler melden	jemand muss ihn sehen

ISR wartet nicht. Ein ADC hält nicht automatisch an.

Notizen

16 / 48

Zeitgleiche Ereignisse und die Belegung vor dem folgenden Get gehören in die Rechnung. Durchschnittsraten allein genügen nicht.

CMSIS-API: Queue anlegen

```
typedef struct {
    uint32_t sample;
    uint32_t seq;
    uint32_t tick;
} SensorMessage;

osMessageQueueId_t q;

void app_init(void) {
    if (osKernelInitialize() != osOK) Error_Handler();
    q = osMessageQueueNew(8, sizeof(SensorMessage), NULL);
    if (q == NULL || osThreadNew(Producer, NULL, NULL) == NULL)
        Error_Handler();
    if (osKernelStart() != osOK) Error_Handler();
}
```

- sizeof und Alignment am Build prüfen
- uint32_t-Übung darf bleiben; das Struct ist der Datensatz
- 1-ms-Modell und osDelay(10) sind verschiedene Versuche

Notizen

osDelay zählt Ticks. Der Thread läuft danach, wenn er dran ist. Periodisch: absolutes Soll und Überlastregel aus VL3.

CMSIS-API: Daten schreiben (Put)

```

void Producer(void *argument) {
    SensorMessage m;
    (void) argument;
    m.seq = 0;
    for (;;) {
        m.sample = next_sample(); /* oder synthetische Folge */
        m.seq++;
        m.tick = osKernelGetTickCount();
        if (osMessageQueuePut(q, &m, 0, 0) != osOK)
            drops++;
        osDelayUntil(next_deadline());
    }
}

```

Status	Bedeutung
osOK	Kopie ist in der Queue
voll, Timeout 0	nichts eingereicht
Timeout	Wartezeit um, nichts eingereicht
Parameter, ISR, Handle	kein „einfach voll“

msg_prio = 0 im Labor: FIFO-Entscheidung, vom FreeRTOS-Adapter ignoriert.

Notizen

Hardware nur anbinden, wenn ADC und Takt schon stimmen. Sonst eine benannte Sequenzquelle.

CMSIS-API: Daten lesen (Get)

```
void Consumer(void *argument) {
    SensorMessage m;
    (void) argument;
    for (;;) {
        if (osMessageQueueGet(q, &m, NULL, 20) != osOK)
            continue; /* alten Wert nicht als neu werten */
        use(m.sample, m.seq);
    }
}
```

- Signatur void Task(void *argument)
- GetCount nur zur Diagnose

Notizen

19/48

Frage: Darf Put blockieren, während ein Mutex gehalten wird, den der Consumer braucht? Nur wenn die Reihenfolge aus VL4 das erlaubt.

Folie

20/48

Unter der Haube: Copy by Value

	Value-Queue	Pointer-Queue
Element	Sample oder Struct	Adresse oder Deskriptor
RAM der Queue	N mal Elementgröße	N mal Zeigergröße
Nutzdaten	liegen in der Queue	liegen woanders, gemeinsam

- Auch der Pointer wird **by value** kopiert
- Das Bild wird nicht kopiert
- Es entsteht kein Eigentumsvertrag
- `sizeof(uint8_t *)` im CM4-Build: 4 Byte, keine Sprachgarantie für jede ABI
- Nach erfolgreichem Put darf die Pointervariable woanders hinzeigen
- Das Array darf erst nach vereinbarter Freigabe neu beschrieben werden

Notizen

20/48

`osMessageQueuePut(q, &image_ptr, 0, 0)` übergibt die Adresse der Pointervariable. Nicht die Bildbytes als Elementspeicher.

Der Feind der Queue: Große Daten!

320 × 240 Byte:

- ein Bild: 76800 Byte = 76,8 kB = **75 KiB**
- zwei Puffer: 150 KiB
- zehn Bilder als Value-Queue: **750 KiB** Nutzdaten, plus Verwaltung
- zehn Pointer-Slots: **40 Byte**, die Bilder bleiben extra

Große Kopien sind teuer, nicht pauschal verboten. Es zählen Größe, Rate, RAM und maskierte Kernelpfade.

Notizen

21 / 48

Keine erfundenen Millionen Zyklen. Kopierzeit messen, wenn ein Build da ist.

Folie

22 / 48

Die Lösung: Copy by Reference (Pointer)

```
static uint8_t buf_a[64], buf_b[64];
uint8_t *slot = buf_a; /* gültiges Ziel, nicht ein leerer Zeiger */
if (osMessageQueuePut(q, &slot, 0, 0) == osOK)
    slot = buf_b; /* Variable umbiegen, buf_a bleibt belegt */
else
    ; /* buf_a bleibt beim Producer, kein zweites Put */
```

- Die Adresse wird kopiert, slot darf danach umbogen werden
- Zwei Arrays A und B allein verhindern das Überschreiben nicht
- Fehlgeschlagenes Put: Puffer bleibt beim Producer
- Zu wenige echte Puffer: die Pointer-Queue läuft nicht von selbst voll mit Bildern

Notizen

22 / 48

Ein Deskriptor mit Länge und Sequenz ist größer als 4 Byte. Das auf der Folie sagen, nicht stillschweigend „vier Byte“ für den Deskriptor.

Gefahren von Copy by Reference

Lebenszyklus, genau ein Verantwortlicher:

FREE → FILLING → QUEUED → PROCESSING → FREE

- Ready-Queue und Free-Pool sind zwei Richtungen
- Put ok: Verantwortung geht weiter
- Put scheitert: keine zweite Einreihung, keine Doppelrückgabe
- Queue für zehn Pointer erzeugt keine zehn Bilder, wenn nur A und B existieren

Fehler in den Notizen: Stack der beendeten Funktion, freigegebener Heap, DMA schreibt weiter.

Notizen

23 / 48

H7-DMA und D-Cache nur, wenn der Puffer wirklich im CM7-Cache liegt. Auf dem CM4 keine pauschalen Cache-Aufrufe. AN4839 nur für diesen Fall.

Folie

24 / 48

Der Spezialfall: Die Mailbox

Drei Abläufe, Start leer, dann 10, 20, 30:

Muster	Ergebnis
FIFO mit Platz	10, dann 20, dann 30
Länge 1, Put Timeout 0	10 bleibt, 20 und 30 werden abgelehnt
Overwrite	zuletzt 30

Länge 1 ist **keine** Overwrite-Semantik.

„Timeout 0 als Näherung“ ist falsch: der alte Wert bleibt.

Notizen

24 / 48

Frage: Ist Put mit Timeout 0 ein Overwrite? Nein. Das Gegenbeispiel ist genau 10 gegen 20.

Wann Queue, wann Mailbox?

„Mailbox“ hier: Latest-Value, Begriff je RTOS anders.

Native FreeRTOS, eigenes Modul, kein Cast auf `osMessageQueueId_t`:

- `xQueueCreate(1, ...)`
- `xQueueOverwrite`
- `xQueuePeek`
- in der ISR die FromISR-Variante und die Yield-Regel

CMSIS-only: kurzer geschützter Snapshot mit Version und Zeit, Benachrichtigung **nach** dem Veröffentlichen. Die ISR nimmt keinen Mutex. Nutzdaten kommen über einen erlaubten kurzen Kanal in den Thread.

Notizen

25 / 48

Peek garantiert keinen neuen Wert und weckt nicht alle Leser. Eine Peek-Schleife ist kein Eventwarten.

Folie

26 / 48

Das Broadcast-Konzept (Mailbox)

- Get: ein Element, **ein** Consumer
- Peek oder Snapshot: mehrere Leser, zu verschiedenen Zeiten verschiedene Versionen
- Jeder Pflichtempfänger braucht einen eigenen Zustellvertrag
- Zusammenfassen von Zuständen kann gewollt sein
- Ereignisse brauchen die FIFO oder eine Zählung

Notizen

26 / 48

Version beantwortet „noch derselbe Wert?“. Sie ersetzt keine Queue.

Zusammenfassung Halbzeit

- Historie: FIFO, Überlauf explizit
- Zustand: Overwrite oder Snapshot, nicht die 1er-Queue mit Try-Put
- Pointer: 40 Byte Slots sind nicht 750 KiB Bilder
- Warten und Fehlerstatus gehören zum Pfad

Notizen

27 / 48

Zweiter Teil: die ISR liefert nur, was sie rechtzeitig retten und erlaubt signalisieren kann.

Folie

28 / 48

Block 4: Interrupts im RTOS

- Die ISR unterbricht Threads **dieses** Kerns
- Höhere IRQs können sich schachteln, je nach Maske
- Der andere Kern und DMA können weiterlaufen
- „Die CPU steht“ ist falsch
- Eine lange ISR verzögert Threads und kann den Tick verschieben
- Verpasste Ticks sind nicht beliebig tief gespeichert
- Ob Fristen reißen, sagt das Budget, nicht der Satz „alle Deadlines wandern“

Notizen

28 / 48

99 % Blocked und „ISR unter 1 μ s“ ohne Messaufbau streichen.

Folie

29 / 48

Rückblick: Die goldene Regel der ISR

- Kurz, keine blockierende API, kein `printf`, kein Mutex
- Timeout 0 macht eine verbotene CMSIS-Funktion nicht erlaubt
- Nicht jede native Funktion hat FromISR
- Falsche Nutzung: Status, Assert, Lockup oder Inkonsistenz, nicht zwingend HardFault

Notizen

29 / 48

IPSR-Prüfung im Wrapper ist ein Implementierungsdetail, keine Garantie für jede API.

Das Architektur-Muster: Deferred Interrupt Processing

Top-Half: das, was die Hardware sofort braucht.

Bottom-Half: der Thread, Priorität nach **Frist und Last**, nicht pauschal „sehr hoch“.

Register lesen und Flags löschen in der Reihenfolge des Geräts. „Erst Flag, dann Daten“ gilt nicht überall.

Im HAL-EXTI-Pfad nicht ein zweites Mal quittieren.

Notizen

30 / 48

Der Worker wird Ready und läuft nach ISR und PendSV, wenn ihn niemand Höherer ausbremst. Die CPU war vorher nicht zwingend Idle.

Folie

31 / 48

Die Top-Half (Die ISR)

- Quittung und die Daten, die sonst verloren gehen
- Ein Semaphor-Token enthält kein Bild
- Drei Releases vor dem nächsten Acquire, max count 1: höchstens ein Token, zwei Releases scheitern
- „Arbeit da“ passt; lückenloses Zählen braucht eine Queue mit Überlaufzähler
- Prellen, Zusammenfassen und Queue-Drop sind drei verschiedene Dinge

Notizen

31 / 48

Liest die Bottom-Half ein Register zu spät, ist das Sample schon weg. Dann muss der Top-Half kopieren oder DMA nutzen.

Folie

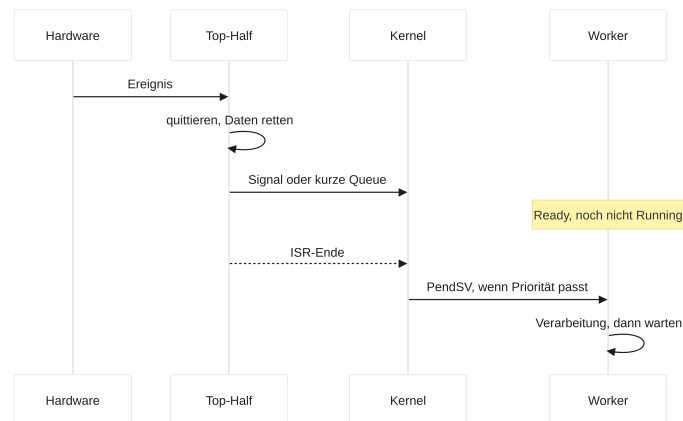
32 / 48

Die Bottom-Half (Die Task)

- Acquire-Status prüfen, dann arbeiten, wieder warten
- Mehrere gepufferte Sätze ohne neue Flanke sind erlaubt
- UART über den geschützten Pfad aus VL4
- Normal: kurze Arbeit und `osDelay` / Warten auf das nächste Signal
- `HAL_Delay(100)` nur als Vergleich: Busy-Wait, High kann Low verdrängen, höhere Tasks können trotzdem präemptieren
- Eine weiterblinkende LED ist kein Fristbeweis

Die Demo friert nicht den ganzen Chip ein.

Visualisierung: Top-Half und Bottom-Half



Kein Thread-Code mitten in der ISR. Ein zweiter Release auf dem vollen binären Semaphor kann scheitern.

Wie signalisiert die Top-Half?

Bedarf	Mittel
Arbeit vorhanden	binärer Semaphor, leer erzeugt
jedes Ereignis oder Nutzdaten	Queue, Timeout 0, Drop zählen
nur ein neuer Stand	Overwrite oder Snapshot, nicht Try-Put

Der Puffer muss gültig bleiben, bis der Vertrag ihn freigibt.

Token ohne gültige Daten weckt den Worker ins Leere.

Die große Hürde: OS-Aufrufe in der ISR

Drei Fragen, alle ja:

1. Ist die Funktion in der ISR erlaubt?
2. Ist der Aufruf nicht blockierend?
3. Liegt die IRQ-Priorität im Syscall-Bereich?

CMSIS	aus ISR
Queue Put/Get	nur Timeout 0
Semaphore Release	ja
Semaphore Acquire	nur Timeout 0
Mutex Acquire/Release	nein
New, Reset, Delete	nein

Notizen

35 / 48

Native Namen daneben nur, wenn das Labor sie wirklich ruft. CMSIS-Labor bleibt bei CMSIS.

Folie

36 / 48

Native FreeRTOS-Lösung: Eigene ISR-Funktionen

- `xQueueSendFromISR`, `xSemaphoreGiveFromISR`
- `xHigherPriorityTaskWoken` beginnt bei `pdFALSE`
- danach `portYIELD_FROM_ISR`
- Fehlendes Yield: unter den Annahmen zuerst zurück zur unterbrochenen Task, bis ein späterer Anlass schedult
- Das ist ein Fehlerbild, nicht „der Kernel wartet immer auf den nächsten SysTick“

Notizen

36 / 48

Folie der alten Fassung, die den Tick als Standard nannte, gilt nur für dieses vergessene Yield.

Die CMSIS-RTOS v2 Abstraktion

- Der ST-Adapter prüft den Kontext und ruft bei erlaubten Funktionen die native FromISR-Variante
- Die Switch-Anforderung sitzt im Wrapper
- Im CMSIS-Labor kein zweites manuelles Yield
- Der Wrapper macht Mutex-Aufrufe dadurch nicht ISR-fähig

Notizen

37 / 48

`cmsis_os2.c` der verwendeten Version lesen, nicht nur die Spezifikation.

Folie

38 / 48

Die eine CMSIS-Regel für ISRs

Kleine NVIC-Zahl = höhere IRQ-Dringlichkeit.

Taskpriorität High repariert eine falsche EXTI-Priorität nicht.

Beispiel, nur bei bestätigten 4 Priority-Bits und Grenze 5: logisch 0–4 keine Kernel-API, 5–15 bei passender Gruppierung erlaubt.

`configMAX_SYSCALL_INTERRUPT_PRIORITY` ist der **geschobene** Registerwert.

Notizen

38 / 48

`__NVIC_PRIO_BITS`, Shift und Grouping aus dem Projekt. „Priorität 5“ nicht universell hinschreiben.

Der Context-Switch aus der ISR (Yielding)

Synthetische Spur, keine H755-Messung:

Ereignis	Zeit
Flanke	0 μ s
ISR-Eintritt	3 μ s
Worker Ready	8 μ s
ISR-Ende	12 μ s
Workerstart nach PendSV	16 μ s

Startlatenz in diesem Modell: **16 μ s**.

Dazu kommen Maskierung, fremde ISRs und die Worker-Laufzeit bis Completion.

Notizen

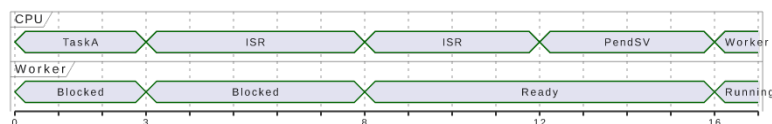
39 / 48

Ready vor ISR-Ende ist möglich. Running als Thread davor nicht. PendSV wartet auf höher dringliche Handler.

Folie

40 / 48

Wie löst FreeRTOS das Yielding?



Hardware-Unterbrechung macht Task A nicht im selben Sinn Ready wie ein Scheduler-Wechsel. A ist verdrängt, solange ISR oder der höhere Worker die CPU hat.

Notizen

40 / 48

Dieselben Zeitpunkte wie in der Tabelle. Nicht aus $t = 1$ bis $t = 3$ auf „minimale Latenz“ schließen.

CMSIS ist unser Freund!

- Erlaubte CMSIS-ISR-Funktionen nehmen das Yield mit
- Unerlaubte bleiben unerlaubt
- Diagnose: Rückgabewert, Assert, Zähler — kein `printf` in der ISR
- Eine IRQ oberhalb der Grenze darf Daten ablegen und eine niedrigere, erlaubte IRQ anstoßen
- Sie darf den RTOS-Aufruf nicht selbst erzwingen
- Das ist Anhang, nicht der Tasterversuch

Notizen

41 / 48

Quelle: FreeRTOS-Kernel-Book Kap. 7 und der CM4F-Port `port.c`.

Folie

42 / 48

Visualisierung: Präemption aus der ISR

Ereignis → ISR → Ready → ISR-Ende → PendSV → Worker.

Nicht: Ereignis → sofortiger Thread.

Die alte kleine Kurve ohne Einheit ersetzt dieses Bild nicht.

Notizen

42 / 48

Completion ist Start plus Arbeit plus weitere Unterbrechungen.

Folie

43 / 48

Laborvorbereitung: Deferred Interrupts

Teil A: Queue mit `uint32_t` oder `SensorMessage`, Sequenz, definierte Drops.

Teil B: User-Button. UM2408 für NUCLEO-H755ZI-Q / MB1363: Standard PC13, Alternative PA0. Gelbe LD2 an PE1, wenn die Brücken so stehen.

GPIO-Takt, Ausgangsmodus und EXTI-Zielkern prüfen. Der RTOS-aufrufende Handler muss im CM4-Kontext liegen.

Notizen

43 / 48

Nur den NVIC-Haken zu setzen, reicht nicht. Priorität muss die Syscall-Grenze einhalten.

Schritt 1: Die Top-Half (ISR) in C

Reihenfolge:

1. Takt und GPIO
2. `osKernelInitialize`
3. `ButtonSem = osSemaphoreNew(1, 0, NULL)`
4. Threads prüfen, `osKernelStart`
5. EXTI erst im laufenden Kernel freigeben

Ausstehende Flags nach diesem Vertrag löschen, nicht im frühen `MX_GPIO_Init` einen Callback ins Leere schicken.

Pfad: `EXTI15_10_IRQHandler` → `HAL_GPIO_EXTI_IRQHandler` → `Callback` → `osSemaphoreRelease`.

HAL quittiert. Der Callback prüft die Leitung und kehrt zurück.

Notizen

`osOK`, voller Semaphor und anderer Fehler getrennt zählen. Kein Heap, keine lange Schleife. Diagnosezähler nicht aus zwei Kontexten unsynchronisiert erhöhen, wenn beide schreiben.

Schritt 2: Die Bottom-Half (Task) in C

```
void ButtonTask(void *argument) {
    (void)argument;
    for (;;) {
        osStatus_t st;
        st = osSemaphoreAcquire(ButtonSem, osWaitForever);
        if (st != osOK) continue;
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) != pressed)
            continue;
        osDelay(20); /* 20 Ticks bei 1 kHz, Pegel erneut prüfen */
        if (HAL_GPIO_ReadPin(GPIOC, GPIO_PIN_13) != pressed)
            continue;
        HAL_GPIO_TogglePin(GPIOE, GPIO_PIN_1);
    }
}
```

Vergleichsvariante, ausdrücklich: `HAL_Delay(100)` als Busy-Wait. Heartbeat darunter kann stocken. Eine noch höhere Task kann unterbrechen.

Entprellung: stabile Pegel und Press/Release im Worker. 100 ms Delay löscht keine schon gespeicherten Folgeevents.

Notizen

PE1 nur, wenn LD2 dort wirklich anliegt. Burst: mehrere Releases vor dem Acquire zeigen das Zusammenfassen. Eine Event-Queue mit Timeout 0 ist die Zählvariante, ebenfalls endlich.

Was werden wir im Labor beobachten?

Feld	Notiz
Semaphor anfangs 0	
NVIC-Zahl und Grouping	
Tickfrequenz	
Workerpriorität	
Drop- und Fehlerzähler	
ISR-Dauer, Start, Completion	
Heartbeat-Verzögerung	
verarbeitete Ereignisse	

Augenmaß an der LED ist keine quantitative Frist.

Messung: GPIO oder vorhandenes Trace, `printf` danach. Pins nur aus dem Projekt.

Notizen

46 / 48

Ohne Board bleiben die Sollbeobachtungen offen. Ein Build beweist keine Latenz.

Folie

47 / 48

Zusammenfassung des Tages

- Variable: letzter Wert; FIFO: Folge, bis der Puffer voll ist
- Ringbeispiel endet bei 20, 30, 40; Head = Tail und Count = 3 heißt voll
- Dauerlast: erster Fehl-Put bei 13 ms, Bilanz 13 und 7
- Pointer-Queue kopiert 4 Byte und nicht das Bild
- Try-Put auf voller 1er-Queue behält 10
- Startlatenz im Lehrbeispiel 16 μ s, nicht gemessen
- EXTI-Priorität und leerer Semaphor gehören zur Init

Notizen

47 / 48

Quellen: CMSIS Message Queue und Semaphore, CMSIS-FreeRTOS Tech Data der Projektversion, FreeRTOS-Buch Kap. 5 und 7, CM4F-Port, UM2408.

Ausblick

- Nächster Termin: mehrere Kerne, gemeinsamer Speicher, kein gemeinsamer Heap
- Die Queue von heute bleibt lokal auf ihrem Kernel
- Cache und DMA nur mit dem Kern, der den Puffer cached

Notizen

HSEM und MPU sind nicht diese Vorlesung.